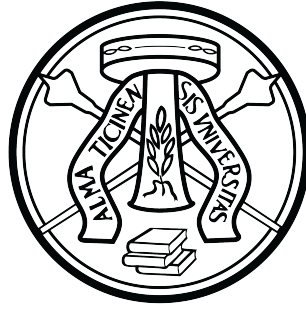




UNIVERSITÀ DI PAVIA

Machine Learning

Euro Coin Image Classification

From a Multilayer Perceptron to Cost-Sensitive Transfer Learning

Claudio Guarrasi

Department of Electrical, Computer and Biomedical Engineering
University of Pavia

Email: claudio.guarrasi01@universitadipavia.it

June 13, 2026

Abstract

This report tackles the classification of euro coin images across the eight denominations, using a dataset of only 800 training images. Three architectures of increasing sophistication are developed in sequence, each motivated by the limitations of the previous one: a multilayer perceptron, a convolutional neural network trained from scratch, and a pretrained ResNet18 adapted through transfer learning. Test accuracy rises from approximately 70% for the MLP to 79% for the CNN and 96% for the fine-tuned ResNet18, confirming that on small datasets reusing features learned on a large dataset clearly outperforms training from scratch. A second objective, minimizing the “value of misclassification” (the mean absolute difference between predicted and true face value), is addressed with a cost-sensitive decision rule that requires no retraining. This rule leaves the confident ResNet unchanged but, on the less certain MLP, lowers the average misclassification cost by steering both its corrections and its residual errors toward coins of nearer value.

Contents

1	Introduction	2
2	Dataset and preprocessing	2
2.1	Dataset	2
2.2	Transforms and data augmentation	2
2.3	Data loading	3
3	Model progression	3
3.1	Multilayer perceptron	3
3.2	Convolutional neural network (from scratch)	4
3.3	Transfer learning with ResNet18	5
3.4	Model comparison	6

4 Cost-sensitive classification	7
4.1 Motivation and theory	7
4.2 Results	8
5 Conclusions	8
6 Limitations and future work	8
Declaration of AI tool usage	9

1 Introduction

This report addresses the task of classifying images of euro coins into their eight classes (1c, 2c, 5c, 10c, 20c, 50c, 1€, 2€). Coin classification is not an easy task because there is a large number of different coin series in circulation, issued in different European Union countries. Each coin series can have numerous variations, such as different legends, mint marks, symbols, and designs, making classification a challenge even for human experts. Each member of the European monetary union has its own version of the coins, with one side representing European unity and the other side featuring a national design. For a more exhaustive discussion of the difficulties of coin classification, see Cirillo et al. [1].

Two objectives are pursued. The first is the standard one: minimizing the classification error. The second is the design of an additional model that minimizes the “value of misclassification”, defined as the mean absolute difference between the predicted and the true face value of the coin (e.g., a 50-cent coin misclassified as a 20-cent coin costs 30 points).

The methodology follows a progression of three architectures of increasing sophistication — a multilayer perceptron (MLP), a convolutional neural network (CNN) trained from scratch, and a pretrained ResNet18 adapted via transfer learning — where each step is motivated by the empirically observed limitations of the previous one. The cost-sensitive objective is then addressed on top of the best model through a decision rule driven by the specification of costs for correct and incorrect predictions, without any retraining.

2 Dataset and preprocessing

2.1 Dataset

The dataset is organized in an `ImageFolder` structure, split into a training set folder and a test set folder. In each of those folder we have one directory per class. The images are RGB PNG files of size 64 x 64 pixels. In the training set folder there are 100 images per class, whereas in the test set folder the images per class are 20 giving a grand-total of 800 images in the training set and 160 images in the test set, making this a small-data image classification problem.

Fortunately the classes are all equally represented, meaning that all the classes will have the same effect on the overall loss. In other terms, the model can “focus its attention” on all classes.

In addition the images have been taken with variable noise, sharpness, lighting conditions, and backgrounds. They can be face up or down, and can be rotated in any direction. With the hope to convey enough variability in the dataset. Examples of train image samples can be seen in Figure 1).

2.2 Transforms and data augmentation

All the images in the dataset were already cropped allowing to have just one coin per image.

The following preprocessing and data augmentation techniques were applied on the images in order to allow them to be suitable for deep learning models and to improve their training phase.



Figure 1: Sample images from the training set.

All images are resized, converted to tensors, and normalized with the ImageNet per-channel statistics in order to move the provided dataset distribution towards the same one the ResNet18 was trained on. The input resolution depends on the model: 32×32 for the MLP and CNN, chosen mainly to lighten the computations of the MLP given its large number of parameters, and 224×224 for ResNet18, matching the resolution at which it was pretrained. Data augmentation is applied to the training set only: random horizontal flips and random rotations. Augmentation artificially increases the effective size of a small dataset and is applied at runtime, so the model never sees exactly the same input twice across epochs. The test set receives only the deterministic preprocessing (resize, tensor conversion, normalization), so that the evaluation reflects the real data distribution.

Rotation is a particularly natural augmentation for coins, which have no canonical orientation in real photographs. However, the appropriate rotation amplitude turned out to depend on the model: aggressive 180 rotations helped the networks trained from scratch, but degraded the pretrained ResNet, whose ImageNet features are orientation-dependent. This is discussed in Section 3.3.

2.3 Data loading

Data is loaded with PyTorch `DataLoaders` using a batch size of 16 with shuffling on the training set. With only 800 training images, a relatively small batch size yields more parameter updates per epoch. In addition because the gradient is computed on a few samples, this gives a poor estimate of the true gradient: a noisier estimate tends to help the model to escape sharp minima (narrow valley in the loss function that tends to overfit the training data) and land on flat minima.

3 Model progression

Three models were developed in sequence. Each subsection describes the architecture, the training configuration, the diagnostic process, and the results; the comparison is summarized in Table 2.

3.1 Multilayer perceptron

For a detailed treatment of multilayer perceptrons (MLP), see [2].

With only 800 images in the training set a huge network (in terms of depth) would overfit immediately. However only a single hidden layer would give the model too little capacity to learn meaningful features combinations from $32 \times 32 \times 3$ input values because the pattern detection capability is limited to just one level of abstraction: the model is forced to go directly to class predictions from raw pixels.

Whereas three or more layers would have too many parameters relative to the data, leading to overfitting. Two layers seems the right balance. Here is the 2-hidden layer MLP structure:

The architecture is a funnel of two hidden layers ($3072 \rightarrow 512 \rightarrow 256 \rightarrow 8$) in order to go from low level features to higher level features with the purpose of classifying correctly the coins. In each hidden block the linear layer is followed by batch normalization, ReLU activation, and dropout. The output layer produces raw logits, since the cross-entropy loss applies the softmax internally. Given the very high ratio of parameters to training samples ($\approx 1.7\text{M}$ parameters for 800 images), the following normalization/regularization mechanisms were employed:

- **Dropout** randomly deactivates a fraction p of neurons at each training step. This prevents the network from relying too heavily on any single neuron and forces it to learn redundant representations distributed across many units, which improves generalization. In this experiment p was set to 0.3
- **Batch normalization** standardizes the output of each linear layer to approximately zero mean and unit variance before the activation. This counteracts the shifting of internal activation distributions during training, stabilizing and accelerating optimization; it also has a mild regularizing effect.
- **L2 weight decay** adds a penalty proportional to the squared magnitude of the weights to the loss, discouraging large weights and thus favoring simpler, smoother functions that are less prone to overfitting. In this experiment the L2 weight-decay coefficient λ was set to 10^{-4} .

Training diagnostics. The initial configuration (Adam optimizer, learning rate 10^{-3}) produced an oscillating training loss and an accuracy plateau around 63%, a typical symptom of a learning rate large enough to repeatedly overshoot the minimum. Lowering the learning rate to 10^{-4} and doubling the number of epochs raised the accuracy to approximately 70%, with reduced but still present oscillation. Adding a `ReduceLROnPlateau` scheduler did not yield further improvement: the loss kept oscillating and the test accuracy did not increase. This indicated that the bottleneck was no longer the optimization but the architecture itself — an MLP discards all spatial structure, treating each pixel independently, and has neither locality nor translation invariance, a limitation made harder to overcome by the small training set.

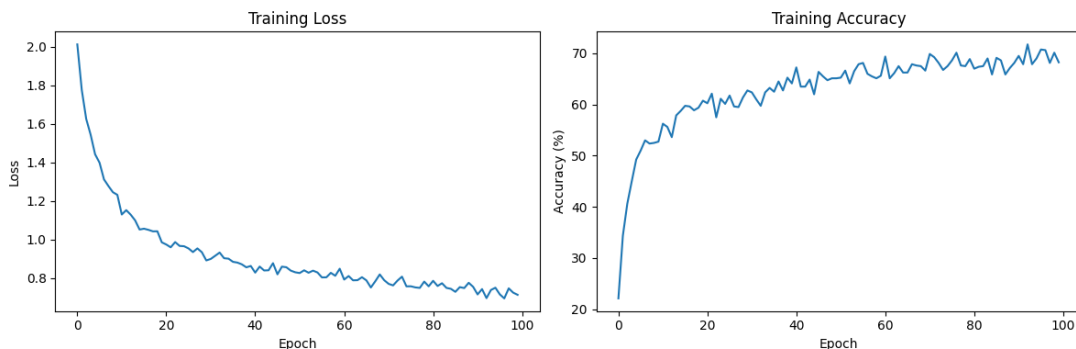


Figure 2: Training loss and accuracy curves, showing clear oscillations.

3.2 Convolutional neural network (from scratch)

For a detailed treatment of convolutional neural networks (CNN), see [2]. The CNN consists of three convolutional blocks, each following the pattern $\text{Conv}(3 \times 3, \text{padding } 1) \rightarrow \text{BatchNorm} \rightarrow$

ReLU $\rightarrow$ MaxPool(2×2), with channel widths doubling across blocks ($32 \rightarrow 64 \rightarrow 128$) as the spatial resolution halves ($32 \rightarrow 16 \rightarrow 8 \rightarrow 4$). A compact classifier head ($2048 \rightarrow 256 \rightarrow 8$ with dropout) completes the network. Despite being a more expressive architecture, the CNN has roughly five times fewer parameters than the MLP ($\approx 345\text{K}$ vs $\approx 1.7\text{M}$), thanks to weight sharing in the convolutional layers.

The number of output channels is doubled at each block in order to compensate for the spatial-dimension shrinkage caused by pooling. The 3×3 convolutions use padding 1 to preserve the spatial size (same-size convolution), so that the role of reducing the resolution is delegated solely to the pooling layers.

Trained with the same loop and hyperparameters as the MLP, the CNN reached 86% training accuracy and 79% test accuracy. The ≈ 7 -point gap indicates mild, acceptable overfitting for a network trained from scratch on 800 images. The training loss still did not converge smoothly, consistent with the small number of batches per epoch (50) and the resulting gradient noise.

The confusion matrix (Figure 3) reflects the model’s accuracy: predictions concentrate on the diagonal. The few remaining errors occur predominantly between visually similar classes, such as the small copper coins (1c, 2c, 5c).

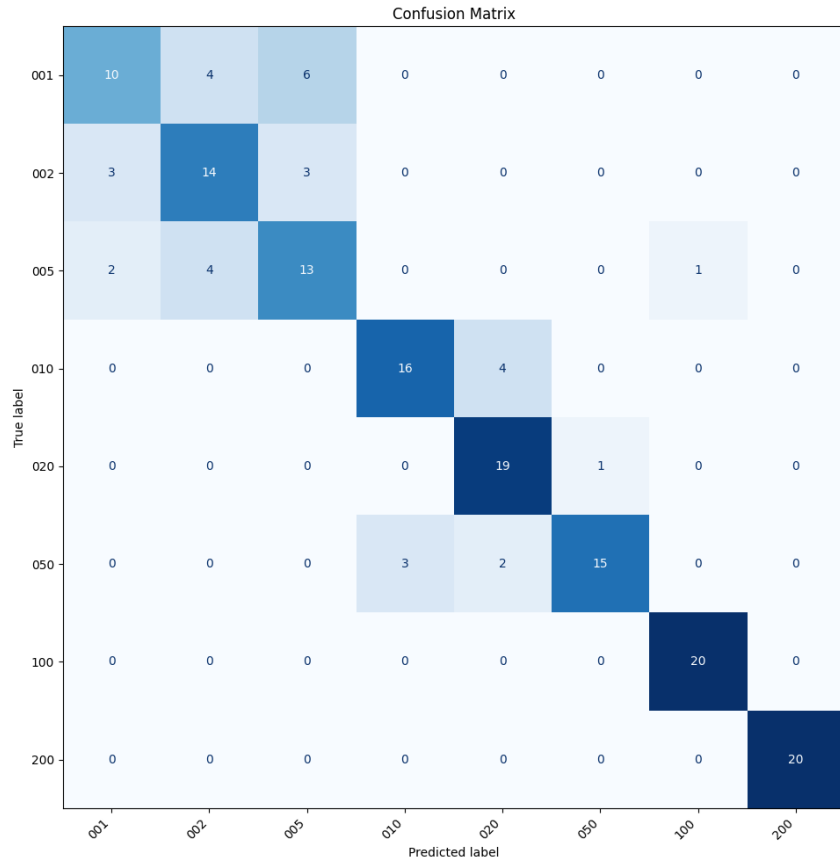


Figure 3: Confusion matrix of the from-scratch CNN on the test set. Rows are true classes, columns are predicted classes.

3.3 Transfer learning with ResNet18

For a detailed introduction to ResNets, see [3].

To overcome the problems of having a small training set preventing a CNN and in general a model from learning meaningful representations from scratch with limited data, we can leverage pre-trained networks. It is a good idea because low and mid level features such as edges, textures

and shapes are universal across image domains, making features learned on huge datasets, like ImageNet dataset, transferable to new tasks.

A pretrained ResNet18 was chosen as the smallest member of the ResNet family. Its residual (skip) connections allow gradients to flow directly across layers, mitigating the vanishing-gradient problem and enabling stable training of deep networks. Among the available depths (18, 34, 50, ...), the 18-layer variant is the most appropriate here: with only 800 training images, a larger backbone would offer more capacity than the data can support and would increase the risk of overfitting during fine-tuning, while also being slower to train. ResNet18 therefore provides a strong pretrained feature extractor at a model size proportionate to the dataset.

Adapting a pretrained network to a new task is known as *transfer learning*, and the process of unfreezing some or all of its weights and continuing training on the target data is called *fine-tuning*. Training proceeded in two phases: the new classification head is first trained alone so that its initially random weights settle to reasonable values, before the whole network is fine-tuned. This ordering avoids large, poorly-directed gradients from an untrained head propagating back and disturbing the valuable pretrained features.

Phase 1 (frozen backbone). All pretrained layers were frozen and only the new final layer ($\approx 4\text{K}$ trainable parameters) was trained with Adam (lr 10^{-3} , weight decay 10^{-4}).

Phase 2 (fine-tuning). All layers were then unfrozen and trained with a much lower learning rate (10^{-4}), so that the pretrained convolutional weights were gently adapted to coin-specific features rather than overwritten.

Training diagnostics. Phase 1 initially underperformed ($\approx 60\text{--}66\%$ accuracy). Two causes were identified. First, an excessive learning rate (10^{-1}) in an early run. Second, and more interesting, the 180° random rotation, which had helped the models trained from scratch, degraded the pretrained features, which are orientation-dependent because ImageNet objects have a canonical orientation. Reducing the rotation to 15° was empirically the best setting (Table 1).

Rotation amplitude	Train accuracy	Test accuracy
$\pm 15^\circ$	82%	77.5%
$\pm 30^\circ$	75%	74%

Table 1: Effect of rotation augmentation amplitude on Phase 1 of transfer learning.

With this configuration, Phase 1 converged to approximately 77% train / 75% test accuracy. Fine-tuning in Phase 2 produced a dramatic improvement to 99% training and 96% test accuracy, with training converging by approximately epoch 40 (motivating early stopping, see Section 6). The 3-point train/test gap indicates minimal overfitting.

3.4 Model comparison

Model	Total params	Trainable params	Test accuracy
MLP	$\approx 1.71\text{M}$	$\approx 1.71\text{M}$	$\approx 70\%$
CNN (from scratch)	$\approx 345\text{K}$	$\approx 345\text{K}$	79.4%
ResNet18 – Phase 1 (frozen)	$\approx 11.2\text{M}$	$\approx 4.1\text{K}$	75%
ResNet18 – Phase 2 (fine-tuned)	$\approx 11.2\text{M}$	$\approx 11.2\text{M}$	96%

Table 2: Summary of the model progression. Note the contrast between total and trainable parameters in Phase 1.

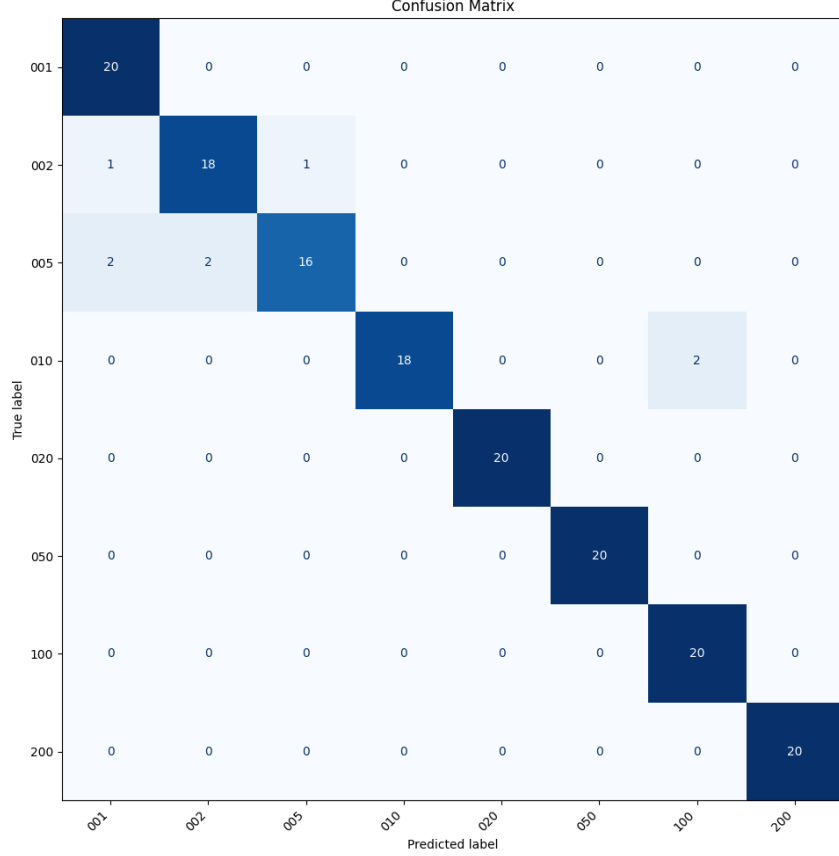


Figure 4: Confusion matrix of the ResNet18 on the test set. Rows are true classes, columns are predicted classes.

The progression illustrates two classical lessons of small-data image classification: convolutional inductive biases beat fully connected architectures at a fraction of the parameter cost, and reusing representations learned on a large dataset dominates training from scratch.

4 Cost-sensitive classification

4.1 Motivation and theory

Classification accuracy treats all errors as equally severe, but in this application misclassifying a 2€ coin as 1c is far more damaging than confusing 10c with 20c. The requested metric is the mean absolute difference between predicted and true face value. This is a cost-sensitive learning problem [4], with cost matrix

$$C(i, j) = |v_i - v_j|, \quad (1)$$

where $v \in \{1, 2, 5, 10, 20, 50, 100, 200\}$ (cents) indexes the eight denominations.

The Bayes-optimal decision rule under a cost matrix replaces the argmax over class posteriors with the minimization of the conditional risk:

$$R(i | x) = \sum_{j=0}^7 P(y = j | x) C(i, j), \quad \hat{y} = \arg \min_i R(i | x). \quad (2)$$

Crucially, the network does not need to be retrained: minimizing the standard cross-entropy already drives the softmax output toward an estimate of the posterior $P(y | x)$, which is exactly the quantity Equation (2) requires. The cost-sensitive model is therefore the same trained

network equipped with a different decision rule at inference time. This has the practical advantage that a single trained model supports any cost matrix, and the two requested models (accuracy-optimal and cost-optimal) share all of their weights.

4.2 Results

With the fine-tuned ResNet18, the risk-minimizing rule made the same predictions as the standard argmax rule, leaving the misclassification cost unchanged. This happens because the cost-sensitive rule can only improve on argmax when the model is uncertain: it then hedges toward a coin whose value is close to the likely alternatives, limiting the damage of a possible mistake. Since the ResNet is highly accurate and confident, it leaves almost no uncertainty to exploit, and the two rules coincide.

To appreciate the mechanism, we therefore evaluate it on the weaker and less confident MLP (Table 3), where the cost-sensitive rule does reduce the average misclassification cost.

Decision rule (MLP)	Mean misclassification cost (cents)
argmax (standard)	4.47
risk-minimizing (cost-sensitive)	3.48

Table 3: Mean absolute face-value error of the two decision rules, evaluated on the MLP.

The improvement comes from the way the rule handles uncertain cases: when the model is unsure, it favors the coin whose value is closest to the likely alternatives. Sometimes this recovers the correct class outright (for instance a 20c coin that argmax had confused with 50c); in other cases the prediction stays wrong but lands on a coin of nearer value, so the error becomes cheaper (for instance a 1c coin confused with 5c rather than with 50c). Either way, the rule lowers the average cost not by guaranteeing more correct answers, but by steering both its corrections and its residual mistakes toward coins of similar value.

5 Conclusions

This work classified euro coin images under two objectives. For classification error, a progression $\text{MLP} \rightarrow \text{CNN} \rightarrow \text{fine-tuned ResNet18}$ raised test accuracy from $\approx 70\%$ to 96% , confirming that on small datasets transfer learning is decisively superior to training from scratch. For the value of misclassification, a Bayes-optimal cost-sensitive decision rule applied to the same trained network reduced the mean absolute face-value error, essentially for free, by trading a small number of cheap errors for the elimination of rare but expensive ones.

6 Limitations and future work

- **Validation methodology.** Hyperparameters were tuned by observing test performance, since no validation split was used. A rigorous protocol would hold out a validation set (or use cross-validation) and touch the test set only once.
- **Early stopping.** Fine-tuning converged by epoch ≈ 40 of 50; an early-stopping criterion with best-model checkpointing would make training more efficient and guard against late-stage overfitting.
- **Dataset size and variability.** With 800 training images, results carry non-negligible run-to-run variance; averaging over multiple seeds would strengthen the conclusions.

Declaration of AI tool usage

In accordance with the assignment guidelines, I declare that AI tools (Claude, Anthropic) were used for the following tasks:

- Debugging code issues, such as loading the dataset from Google Drive and diagnosing the cause of training instabilities (e.g. the oscillating loss and the Phase 1 underperformance of the transfer-learning model).
- Discussing and clarifying machine learning concepts, including the choice of batch size, regularization mechanisms, data augmentation, and the theory of cost-sensitive classification.
- Assisting with the drafting, structuring, and wording of this report, including the generation of the L^AT_EX template.

All design decisions, experiments, results, and their interpretation are my own, and the content has been reviewed and validated by me.

“I affirm that this report is the result of my own work and that I did not share any part of it with anyone else except the teacher.”

References

- [1] Cirillo, S., Solimando, G. & Virgili, L. A deep learning approach to classify country and value of modern coins. *Neural Computing and Applications* **36**, 11759–11775 (2024). <https://doi.org/10.1007/s00521-023-09355-6>
- [2] Murphy, K. P. *Probabilistic Machine Learning: An Introduction*. MIT Press (2022). <http://probml.github.io/book1>
- [3] He, K., Zhang, X., Ren, S. & Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778 (2016).
- [4] Elkan, C. The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, vol. 17, no. 1, 973–978 (2001).